

Builders Stream — Requirements vs. Implementation Comparison

Date: 2026-07-12 **Source document compared:** Builders Stream (Consolidated Requirements).pdf (v1.0, 2026-07-07, 45 pages) **Scope of implementation reviewed:** Backend (FastAPI), through Phase 2 (Estimation Engine + E-Signature) of the roadmap in that document.

Methodology note: Phase 2 (Estimation Engine, E-Signature, Change Orders) findings below are based on direct implementation and verification work performed this session — high confidence. Phase 0/1 (Users & Company Management CRM Project Management foundation) findings are based on session context/summary plus several targeted direct file reads (`app/core/security.py` , `app/routers/companies.py` , `app/routers/invitations.py` , `.github/workflows/backend-ci.yml`) — good confidence but not as exhaustively re-verified as Phase 2. Frontend UI completeness is **not verified** — flagged explicitly where relevant.

1. Overall Assessment

The database schema, RLS/tenant-isolation architecture, estimation calculation pipeline, and e-signature workflow match the consolidated requirements PDF almost line-for-line. This is a high-fidelity implementation of the documented domain model and security architecture. The deviations found are concentrated in: auth token lifecycle, backend package organization, a handful of API-spec gaps (mostly gaps in the *written spec itself*, correctly filled by the implementation), tooling substitutions, and CI gate completeness.

2. High-Fidelity Matches (Verified)

Area	PDF Requirement	Implementation	Confidence
Company/User schema	<code>companies</code> (nested <code>parent_id</code>), <code>users</code> , <code>company_users</code> (role CHECK: <code>admin/project_manager/field_crew/accountant/client</code>), <code>invitations</code>	Matches exactly, incl. 7-day invitation TTI (<code>INVITATION_TTI_DAYS = 7</code> in <code>app/routers/invitations.py</code>)	High (verified directly)
Lead status enum	<code>new/contacted/estimating/qualified/won/lost</code>	Matches exactly	High (session context)
Project status enum	<code>draft/pre_construction/active/suspended/completed/archived</code>	Matches exactly (<code>Project.VALID_STATUSES</code>)	High (verified directly)
Estimate status enum	<code>draft/sent/approved/rejected</code>	Matches exactly (<code>Estimate.VALID_STATUSES</code>)	High (verified directly)
Change Order status enum	<code>pending/approved/rejected</code>	Matches exactly (<code>ChangeOrder.VALID_STATUSES</code>), including no sign-restricting CHECK on <code>cost_delta</code> (explicitly confirmed — a Change Order can be a credit or an add)	High (verified directly)
Esignature schema	<code>signer_name</code> , <code>signer_email</code> , <code>signed_at</code> , <code>ip_address</code> (INET), <code>signature_artifact_path</code> , <code>document_type</code> CHECK (estimate/change_order)	Matches exactly, incl. the INET→Python-str TypeDecorator fix and REVOKE-based immutability	High (verified directly)
Historical immutability	<code>unit_rate_snapshot</code> separate from live <code>unit_rate</code> ; <code>is_snapshotted</code> locks an Estimate once approved	Matches exactly; empirically proven against a live catalog-price change	High (verified directly)
RLS enforcement	<code>get_all_descendant_ids()</code> , per-table <code>tenant_isolation</code> policy, parent/child hierarchy visibility, sibling isolation	Matches exactly; tested more rigorously than the PDF's own minimum bar (RLS-disable/re-enable proofs applied to every new table shape, not just once)	High (verified directly)
Estimation calculation order	<code>qty×rate → category subtotal → overhead → profit → tax(=0) → single final rounding (Decimal, ROUND_HALF_UP)</code>	Matches exactly	High (verified directly)
E-signature capture	signer name/email, server-captured timestamp+IP, immutable record, shared between Estimate and Change Order approval	Matches exactly, including documenting the X-Forwarded-For	High (verified directly)

Area	PDF Requirement	Implementation	Confidence
		production-IP gap the PDF itself doesn't raise	
Async PDF export	202 Accepted, background job, Dramatiq+Redis	Matches exactly	High (verified directly)
Tenant-isolation test discipline	5 required test cases (Test Strategy §2), incl. "disable RLS in test setup" proof	Implemented essentially verbatim, applied to every new table (not just the PDF's minimum)	High (verified directly)
Audit logging	company_id/actor_id/action/entity_type/entity_id/metadata JSONB, append-only	Matches exactly, used pervasively	High (verified directly)

3. Correctly Out of Scope (Not Gaps)

Per the PDF's own Roadmap (Phases 3–5), none of the following exist yet, and none should at this point:

- Accounting & Billing (Subscription, Invoice, Payment, Expense) — Phase 3
- Compliance Tracking (Subcontractor, ComplianceDocument, expiry notifications) — Phase 3
- Stripe subscription billing / pricing tiers — Phase 3
- QuickBooks / FreshBooks integration — Phase 4
- Offline/PWA mobile support, AI blueprint takeoff, multi-currency — Phase 5 (unscheduled)

4. Real Deviations Worth Attention

4.1 Auth model is simpler than specified

PDF (Security & Compliance §1) calls for OIDC/IWT (e.g. Keycloak or Better Auth) 15-minute token lifetime, revocable refresh-token rotation. Verified directly (`app/core/security.py` , `.github/workflows/backend-ci.yml`):

- Plain HS256 JWT, symmetric shared secret — not OIDC.
- No refresh token mechanism at all.
- No server-side revocation (a `jwt` claim is generated but nothing checks it against a blocklist).
- Token expiry is **60 minutes**, not the PDF's suggested ~15 minutes.

Functionally fine for current scope; a real gap against the written security plan.

4.2 No MFA/TOTP

PDF requires TOTP for the Admin role at minimum. Not present anywhere in the codebase.

4.3 Backend isn't organized as domain-bounded packages

PDF (Technical Architecture §3) prescribes `src/{core,users,crm,projects,estimation,billing,integrations,compliance}/` bounded-context packages with an enforced "no cross-module table queries" encapsulation rule.

Actual layout is a conventional layered FastAPI structure: `app/models/` , `app/routers/` , `app/services/` , `app/schemas/` , `app/core/` — organized by technical layer, not domain package. Domain separation exists in file *naming* (e.g. `estimate.py` , `change_order.py`) but there's no enforced package boundary. Reasonable for a solo-developer project at this size, but a real architectural departure from what's documented.

4.4 API Specification chapter has gaps the code correctly filled

The PDF's own API Specification table (§5) is missing routes its own Functional Requirements chapter requires:

- **No `/estimates/{id}/approve` , `/reject` , `/change-orders/{id}/approve` , `/reject` routes are listed anywhere** in the API spec, even though US-4.5/US-3.6 explicitly require a client to approve-or-reject with e-signature. The implementation correctly built all four routes, reasoning "the spec describes API contracts conceptually, not exhaustively" — defensible, but the written API spec itself is incomplete, not just the implementation diverging from it.
- Also missing from the PDF but correctly added by the implementation: `GET /estimates` (list), `GET /estimates/{id}` , `GET /markup-profiles` (list), `GET /projects/{id}/change-orders` (list).

4.5 Route naming: kebab-case vs. the PDF's literal spelling

PDF writes `/projects/{id}/dailylogs` and `/changeorders/{id}/...`; implementation uses `/projects/{id}/daily-logs` and `/change-orders/{id}/...`. Consistent and sensible, but a literal deviation from the written spec.

4.6 Invitation accept route uses the invitation's own ID, not a distinct token

PDF implies `/invitations/{token}/accept` — a distinct opaque token. Actual implementation is `/invitations/{invitation_id}/accept`, reusing the invitation's own UUID as the accept-URL identifier. Not a practical security issue (UUIDs aren't guessable), but not literally what's written.

4.7 Missing route: GET /companies/{id}/users

Confirmed directly — `app/routers/companies.py` only implements `GET /companies/{id}` and `POST /companies/{id}/children`. A real, small gap against the PDF's API spec (§2).

4.8 PDF-generation library substitution: WeasyPrint → xhtml2pdf

Deliberate, user-approved mid-session change. WeasyPrint's native Pango/Cairo/GDK-PixBuf dependencies don't install cleanly on the Windows dev machine used for this project. The consolidated requirements PDF doesn't literally mandate WeasyPrint (it just says "PDF proposal generation" as an async job), but the phase-2 implementation plan that was actually built against did, until this substitution.

4.9 E2E testing is a custom httpx script, not Playwright

PDF's Test Strategy §1 specifies Playwright for the End-to-End test layer. What actually exists (`scripts/e2e_smoke_test.py`) is a hand-written synchronous-HTTP smoke test run against a live Docker Compose stack. It's functional and has caught real regressions, but it is not the tool named in the plan.

4.10 CI gates are narrower than specified

PDF's CI Gates section (Test Strategy §8) requires unit+integration tests, the tenant-isolation suite, **linting/type-checking (mypy/ruff for backend, tsr/eslint for frontend)** and an OpenAPI schema-diff review, before merge to `main`. Confirmed directly (`.github/workflows/backend-ci.yml`): CI runs only `pytest -v`. No lint step, no type-check step, no frontend CI at all, no schema-diff step.

4.11 Frontend implementation depth — not verified

All direct implementation work this session was backend-only. Confirmed: a Next.js app exists with at least a working health-check page (verified via the E2E script asserting "Backend status: ok" renders correctly), and a separate `marketing-site/` / `marketing/` presence exists beyond the internal app. **Not verified:** whether actual UI screens exist for driving the CRM / Project Management / Estimation workflows described in the Functional Requirements (creating a Lead, building an Estimate, the client e-signature approval flow, etc.). This is the largest open question in this comparison — recommend a dedicated frontend audit if UI completeness matters for your next decision.

4.12 PROJECT_COMPLETED event is never published

The PDF's event-bus table (Technical Architecture §4) lists `PROJECT_COMPLETED`, published by Project Management, consumed by Billing/Integrations. No code path publishes this event anywhere, even though Project Management (including the Task 2.23 completion-blocked-by-open-Change-Orders check) is fully built. Likely fine — its only documented consumer (Billing) is Phase 3 and doesn't exist yet — but worth wiring when Phase 3 begins rather than discovering the gap then.

5. Suggested Next Steps

- If frontend UI completeness matters for your next planning decision, audit `frontend/app/` directly against the Functional Requirements user stories (US-1.x through US-4.x) to confirm actual screens exist, not just the scaffold + marketing pages.
- Decide whether the auth-model gaps (§4.1, §4.2) need addressing before this handles real subscriber data, or whether they're acceptable through the rest of MVP development.
- Consider whether to formally amend the consolidated requirements PDF's API Specification chapter to include the approve/reject routes and list routes that are already correctly implemented but undocumented there (§4.4) — closing the loop between spec and code.
- Add a lint/type-check CI gate (§4.10) before this scales past solo-developer review capacity.